# CollabSpace

Dedicated Space for Creatives

Collaborative Media Management System

Titus Duncan

Spring 2026

# Table of Contents

Cover Page

1.0 Project Proposal

2.0 Customer Problem Statements

2.1 Customer Problem Statement

2.2 Glossary Terms and Visual Representation

2.2.1 Information Structure Examples

2.3 System Requirements (Functional & Non-Functional)

2.4 User Interface Requirements

2.5 Plan of Work Summary

3.0 Functional Requirements Specification

3.1 Stakeholders

3.2 Actors and Goal

3.2.1 Primary Actors

3.2.2 Secondary Actors

3.3 Use Cases

3.3.1 Casual Descriptions + Estimates

3.4 Use Case Diagram

3.5 Class Diagram

4.0 System Sequence Diagram and Activity Diagram

4.1 Sequence Diagrams

4.1.1 Sequence Diagram 1: Add Version (includes Upload Asset)

4.1.2 Sequence Diagram 2: Approve Version

4.2 Activity Diagrams

4.2.1 Activity Diagram 1: Add Version (includes Upload Asset)

4.2.2 Activity Diagram 2: Approve Version

5.0 Traceability Matrix

Additional Documents (Submitted Separately)

User Interface Specification (Landscape PDF for Readability)

## 1.0 Project Proposal

Problem Statement:

Creative work is often produced by small teams and reviewed by clients through a messy mix of tools (shared drives, text messages, email threads, scattered links). This creates recurring issues: teammates overwrite files or lose the "latest" version, feedback gets separated from the correct deliverable, approvals are unclear, and clients struggle to review work in an organized way. These problems cause delays, duplicate work, and frustration for both creatives and clients. **This project addresses the need for a single system that supports collaboration, version tracking, structured review, and clean client-facing presentation of creative deliverables.**

System Objectives:

**The objectives of the system are intended to make creative collaboration and client review more organized, transparent, and efficient.** For creative teams, it will provide a central place to manage projects, upload assets, track version history, and keep feedback attached to the correct deliverable. It will reduce confusion about which file is current and prevent repeated rework caused by missing or scattered comments. For clients, it will provide a simple "presentation page" experience where work is organized in the right order, easy to view, and easy to approve or request changes. The system will also help teams monitor project status (what is in review, what needs changes, what is approved/final) so delivery timelines become more predictable.

Typical Users:

The system will have different features based on the roles of the user.

- Project Owner / Admin (studio owner, team lead)
- Creatives / Contributors (designers, photographers, videographers, editors, producers)
- Clients (viewers/approvers who review deliverables and provide feedback)

System Requirements & User Actions:

The system will include role-based accounts and project workspaces where assets (creative projects) are uploaded, organized, versioned, and presented to clients for review.

Users can:

- Create an account, sign in, and access role-based features (admin/creative/client).
- Create projects and manage project membership (add/remove collaborators).
- Upload media assets (image/video/audio/document) to a project and store metadata (title, description, tags, type).
- Create new versions of an asset and track status (Draft → In Review → Changes Requested → Approved/Final).
- Search and filter assets by keyword, tag, project, type, status, and date.

- Create client presentation pages (curated, ordered deliverables with captions).
- Share presentations with clients through access-controlled links (permissions and optional expiration).
- Clients can view deliverables, leave comments tied to specific versions, and approve or request changes.
- Team can track overall progress (pending reviews, requested changes, approved finals).

Project Planning & Development Approach:

The project will be developed as a database-driven web application. All user actions (uploading assets, tagging, version changes, comments, approvals, and presentation creation) will be stored in the database and reflected through the backend.

1. Software
   - Front-end: HTML/CSS + JavaScript (Django templates for pages, light JS for smoother interactions)
   - Back-end: Python + Django
   - Database: SQLite for development (fast setup), with an option to use PostgreSQL if deployed
   - Code editor: Visual Studio Code
   - Version control: Git + GitHub
   - Key libraries/features: Django authentication/permissions, file upload handling, database ORM queries for search/filtering

2. Hardware
   - A standard PC/laptop capable of running Python and a local database
   - Storage space for uploaded media files (stored locally during development)
   - Optional: a small cloud server if deploying publicly (not required)

3. Network
   - Internet access for installing dependencies and using GitHub
   - For local development and demos, standard internet is sufficient
   - No special cabling requirements

Development Plan:

- w1 – w2: Finalize scope and user roles; set up tools (VS Code, Git/GitHub); create Django project structure; design database (ERD + tables); connect backend to database
- w3 – w5: Build core system: login/roles, projects + team membership, upload assets + metadata/tags, search/filter library
- w6 – w7: Add collaboration workflow: versioning (draft → review → final), comments, and approval/changes-requested flow
- w8: Test core features and record the mid-term demo
- w9 – w11: Build client presentation features: presentation pages, share links with permissions/expiration, client review experience

- w12 – w14: Improve usability, fix bugs, add basic dashboard/status tracking, write test cases, and run a small usability test
- w15: Final testing, documentation, and final demo recording/presentation

## 2.0 Customer Problem Statements

### 2.1 Customer Problem Statement:

"As a smaller-market creative, I don't have a dedicated tool to manage projects and clients. I'm just trying to figure it out, so I use what's common and easy; things like Google Drive folders, texts, emails, and whatever link-sharing works in the moment. But when my work is spread across different tools, organization falls apart. Files get buried, overwritten, corrupted, or lost. I send one version, then later realize the client reviewed a different one. Feedback ends up scattered across messages, and approvals are unclear. I waste time hunting for the latest deliverable instead of creating, and it makes me look less professional than I actually am.

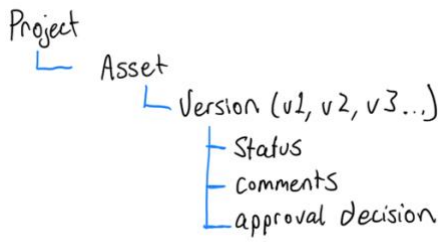
What I expect from a software system is simple: keep everything organized on one dedicated platform. I want one devoted workspace per project where I can upload and label my assets, track revisions, and know exactly what version is current. I want client feedback attached to the correct file, and I want a clean presentation page I can share so clients can review deliverables in order and clearly approve or request changes. If a system can make it simple to organize—so nothing gets lost and everyone stays on the same page—then collaboration becomes smoother, revisions become faster, and delivering final work becomes much more reliable."

### 2.2 Glossary Terms and Visual Representation:

| Term | Definition |
|---|---|
| **Project** | A dedicated space that keeps all work for one job or collaboration together in one place. |
| **Collaborator** | A person invited into a project to contribute work, revisions, or feedback. |
| **Client** | The person the work is for. In some projects, the client may also be a collaborator. |
| **Asset** | A file saved in a project (image, video, audio, or document). |
| **Version** | A saved revision of an asset (v1, v2, v3…) so earlier work is not overwritten or lost. |
| **Current Version** | The version that should be treated as the main one for review or delivery at that time. |
| **Status** | The stage a version is in: Draft, In Review, Changes Requested, or Approved/Final. |
| **Comment** | Feedback attached to a specific version, so notes stay connected to the correct file. |
| **Presentation Page** | A client-facing page that displays selected deliverables in an organized order for review. |
| **Share Link** | A controlled access link to a presentation page (view-only or comment/approve, optional expiration). |

2.2.1 Information Structure Examples:

Project Structure




Project in Review Flow

Client Presentation Flow

## 2.3 System Requirements (Functional & Non-Functional):

| Key Term | Meaning |
|---|---|
| **Req-#** | **Functional Requirement** |
| NFR-# | Non-functional Requirement |
| High | the core promise of "organization is key" (projects, assets, versions, current status, feedback). |
| Medium | review + presentation layer (important, but the system can still function as an organized collaboration vault without all client-facing features complete). |
| Low | search/filtering (nice-to-have early; becomes critical later as volume grows). |

*System Requirements*

| No. | **Priority Weight** | **Description** |
|---|---|---|

| | | |
|---|---|---|
| REQ-1 | High | Users need to create an account to sign in and have access. |
| REQ-2 | High | The system must support **projects** so work stays organized by job or collaboration. |
| REQ-3 | High | A project owner must be able to invite and manage **collaborators** for each project. |
| REQ-4 | High | The system must support uploading **assets** into a project. |
| REQ-5 | High | The system must support **versioning** so revisions don't overwrite or replace older work. |
| REQ-6 | High | Each version needs a **status** (Draft, In Review, Changes Requested, Approved/Final) and a clear **current version**. |
| REQ-7 | High | **Comments** must attach to a specific version so feedback stays tied to the right file. |
| REQ-8 | Medium | **Approval decisions** must be recorded on a version (approve or request changes) to prevent confusion. |
| REQ-9 | Medium | Assets should include organizing details (title, description, tags, type) so projects don't turn into messy file dumps. |
| REQ-10 | Medium | The system should support a **presentation page** that shows selected deliverables in a clear order. |
| REQ-11 | Medium | A **share link** should provide controlled access to a presentation page (view-only or comment/approve). |
| REQ-12 | Low | Search and filters (keyword/tag/type/status) are helpful for speed as projects grow. |
| | | |
| NFR-1 (Usability) | High | The system must feel simple and organized for first-time users, with clear labels and an obvious flow from project → asset → version → feedback/approval. |
| NFR-2 (Reliability) | High | The system must protect work from being lost or overwritten by saving assets, versions, comments, and approvals consistently and keeping version history intact. |
| NFR-3 (Functionality) | High | Access control must be dependable so only invited collaborators and shared clients can view a project or presentation page. |
| NFR-4 (Performance) | Medium | Common actions (opening a project, viewing an asset/version, loading a presentation page) should feel responsive for typical class-scale use. |
| NFR-5 (Usability) | Medium | The system should be easy to navigate on a laptop screen without training, including readable layouts for reviewing deliverables and leaving comments. |
| NFR-6 (Supportability) | Medium | The project must be easy to maintain and grade: clear folder structure, readable naming, and consistent terms across the UI, database, and documentation. |
| NFR-7 (Supportability) | Medium | The system should be easy to set up on a new machine using documented steps (install, configure, run, test). |
| NFR-8 (Performance) | Low | The system should handle moderate growth (more projects/assets) without needing major redesign, as long as users follow basic organization (tags, projects, versions). |

## 2.4 User Interface Requirements:

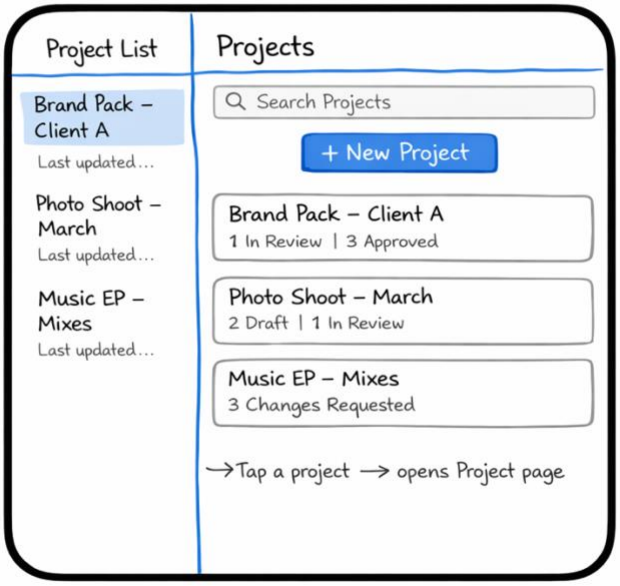
*Disclaimer: I hand drew these images on my iPad, then enhanced them using AI.*

## UI-1: Sign In (High)

Users need to create an account to sign in and have access.

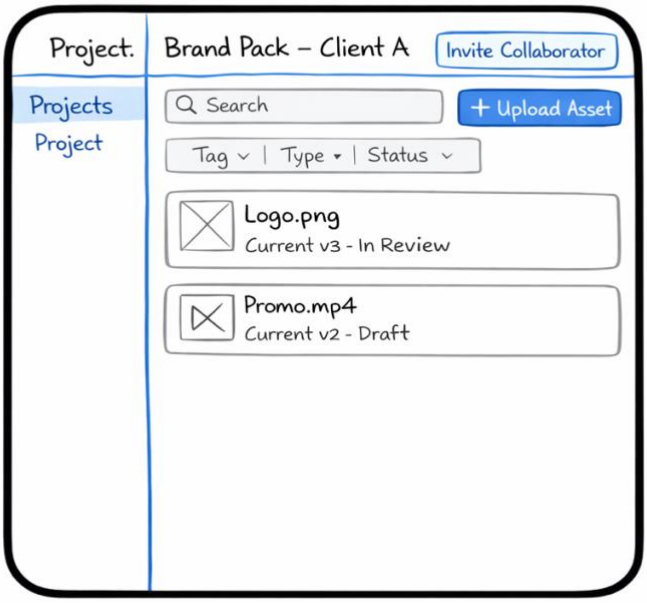
## UI-2: Project Dashboard (High)

After sign-in, users need a dashboard that keeps projects organized and easy to enter.



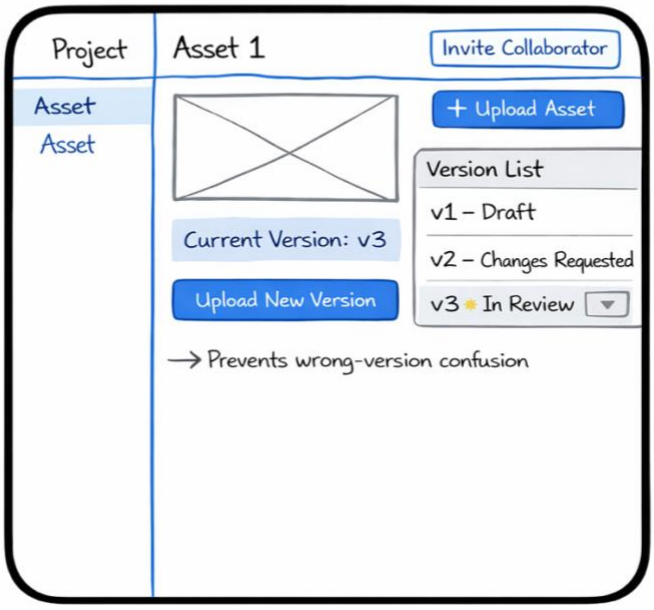
## UI-3: Project Library (High)

Inside a project, users need to upload assets and browse everything in one place.



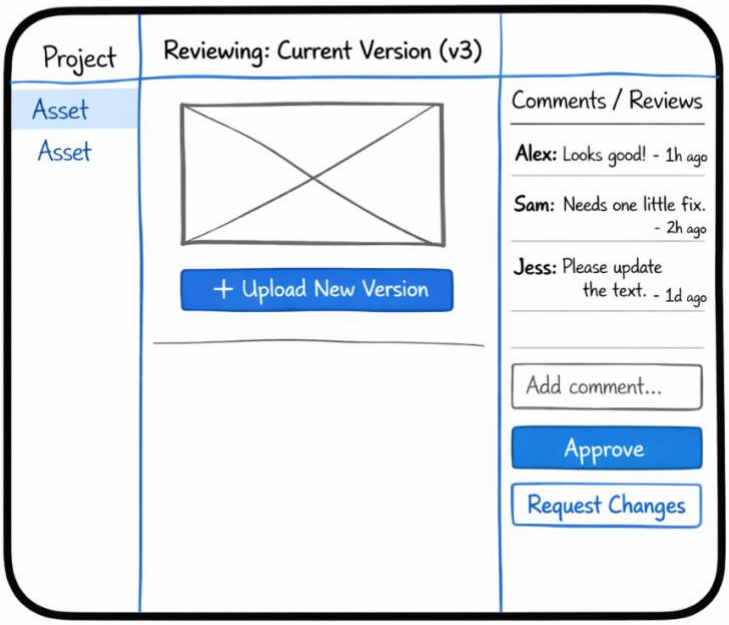

## UI-4: Asset + Versions (High)

Users need a clear asset page that makes the current version and past versions obvious to prevent confusion.



## UI-5: Comments + Approval (High)

Users need to review an asset version and make approval decisions.




## UI-6: Presentation Page (Medium)

Users need to share a clean page for client review.

**2.5 Plan of Work Summary:**

So far, I've completed the following:

- Project Topic
- Customer problem statement
- Glossary of terms
- Functional and Non-functional requirements
- User Interface Requirements
- Basic Sketches

I am currently in the process of:

- Setting up development environment
- Creating Django project structure
- Starting database design

My next focus is:

- Finish environment setup and assure that everything runs local properly
- Build the first working flow as mentioned in section 1.2.1
- Begin database models and connect the UI flow to database actions

## 3.0 Functional Requirements Specification

### 3.1 Stakeholders:

1. **Creators (project owners)**: need an organized home for projects, versions, collaborators, and client delivery.
2. **Collaborators (creative teammates)**: need controlled access to upload, version, comment, and track progress.
3. **Clients (guest or account)**: need simple viewing and clear ways to approve or request changes.
4. **Admin / platform maintainer (future scale)**: needs to manage users, reports, and limits.

### 3.2 Actors and Goals:

3.2.1 Primary Actors

- **Creator**: starts projects, organizes work, manages collaborators, controls "current version," and publishes presentations.
- **Collaborator**: contributes work (assets/versions), updates status, comments, and searches assets—same workflow as creator, but with fewer permissions.
- **Client**: views presentation, uses controlled access, approves a version, or requests changes.

3.2.2 Secondary Actors

- **Admin**: manages users, handles reports, manages platform limits.
- **System**: stores project data, enforces access, maintains "current version," supports search/retrieval.

### 3.3 Use Cases:

1.3.1 Casual Description + Estimates

| Use Case (Creator) | Casual Description | Estimate (pts) |
|---|---|---|
| **Create project** | Start a new project with a title and description so all work stays in one organized place. | 2 |
| **Manage collaborators** | Invite collaborators, assign roles/permissions, and remove access when needed. | 4 |
| **Set current version** | Choose which uploaded version is the official "current" version shown to others. | 3 |
| **Create presentation page** | Build a client-facing page by selecting assets/versions and arranging them for viewing. | 4 |
| **Create account** | Sign up so the creator can access and save projects. | 2 |
| **Login/Logout** | Sign in/out to access projects securely. | 2 |
| **Upload asset** | Add a new asset into a project (file + basic info). | 2 |

| | | |
|---|---|---|
| **Add version** | Upload a new version tied to an existing asset and keep version history organized. | 3 |
| **Update status** | Mark progress (draft/in review/approved/changes requested) so the team knows what's going on. | 2 |
| **Comment on version** | Leave feedback connected to the exact version being discussed. | 2 |
| **Search/filter assets** | Search assets by keyword/tags/type to quickly find what's needed. | 2 |
| **Creator total** | | **28** |

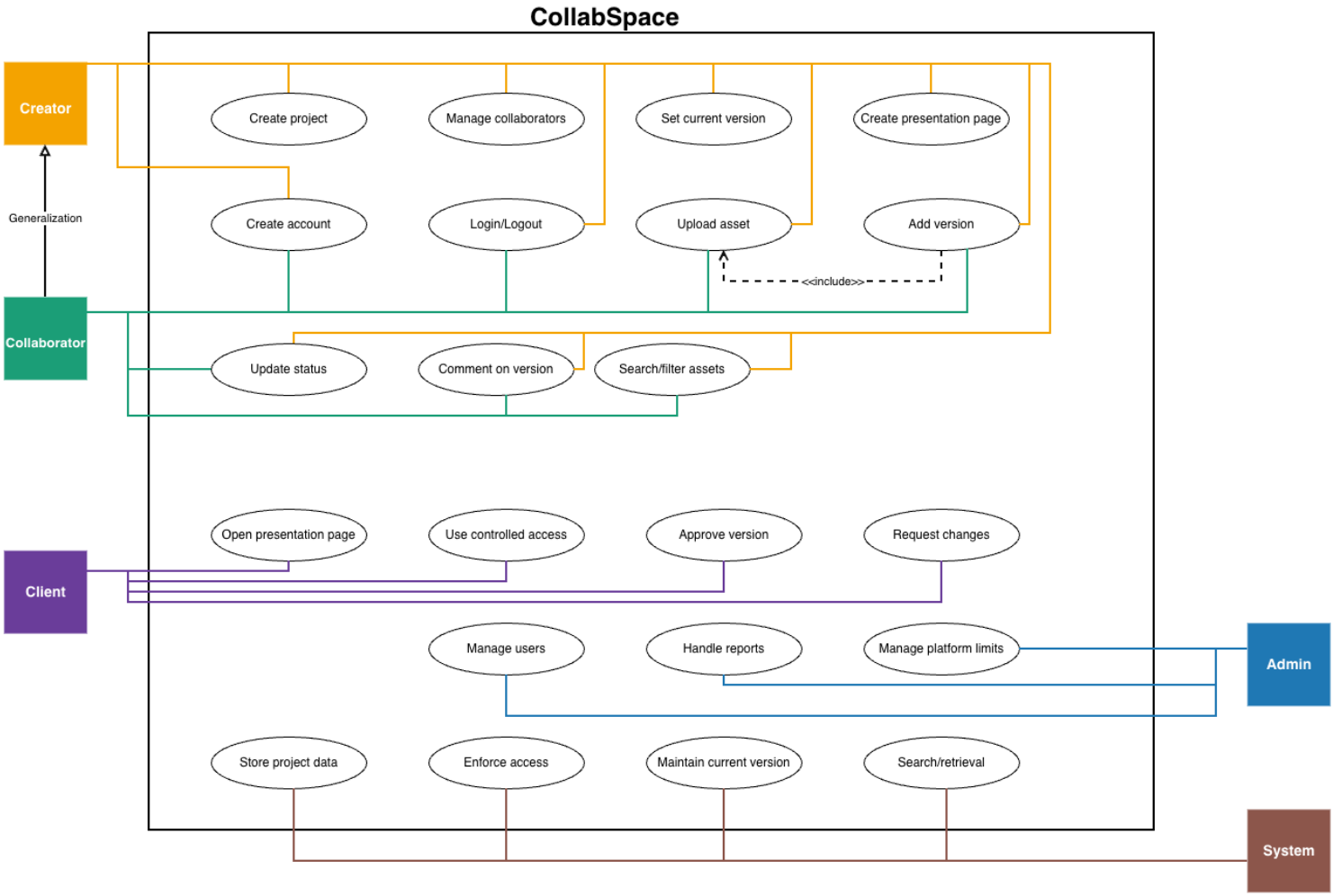
| Use Case (Collaborator) | Casual Description | Estimate (pts) |
|---|---|---|
| **Create account** | Sign up to join projects and contribute work. | 2 |
| **Login/Logout** | Sign in/out to access assigned projects. | 2 |
| **Upload asset** | Add a new asset into the shared project (with permissions). | 2 |
| **Add version** | Upload a revision to an existing asset so progress is tracked cleanly. | 3 |
| **Update status** | Update progress labels so the team stays aligned. | 2 |
| **Comment on version** | Give feedback tied to a specific version. | 2 |
| **Search/filter assets** | Find assets fast without digging through folders/links. | 2 |
| **Collaborator total** | | **15** |

| Use Case (Client) | Casual Description | Estimate (pts) |
|---|---|---|
| **Open presentation page** | View the project presentation in a clean, client-friendly format. | 2 |
| **Use controlled access** | Enter through a secure link or account so access stays intentional and limited. | 3 |
| **Approve version** | Approve a specific version so the team knows what is accepted. | 3 |
| **Request changes** | Request changes on a specific version with notes so feedback stays organized. | 3 |
| **Client total** | | **11** |

| Use Case (Admin) | Casual Description | Estimate (pts) |
|---|---|---|
| **Manage users** | View users and change account status when needed (active/suspended/closed). | 3 |
| **Handle reports** | Review and resolve reports (abuse, broken links, inappropriate content). | 3 |
| **Manage platform limits** | Set basic platform limits (storage, project caps) to support scaling. | 3 |
| **Admin total** | | **9** |

| Use Case (System) | Casual Description | Estimate (pts) |
|---|---|---|
| **Store project data** | Save projects, assets, versions, comments, and reviews reliably. | 3 |
| **Enforce access** | Check permissions and link rules so only allowed people can view or act. | 3 |
| **Maintain current version** | Ensure the selected "current" version is always the one shown by default. | 3 |
| **Search/retrieval** | Return search/filter results quickly based on keywords/tags/type. | 2 |
| **System total** | | **11** |

## 3.4 Use Case Diagram:



### 3.5 Class Diagram:

### Diagram 1 (Accounts/Roles)

This diagram shows the main Phase 1 classes and the direction of their relationships (navigation). A MemberAccount creates/owns a Project. A Project contains Assets, and each Asset has multiple Versions. A Project publishes a PresentationPage, which uses ControlledAccess to control viewing permissions. A ClientAccount views the PresentationPage and can approve a Version or request changes. An AdminAccount manages Accounts (user administration).

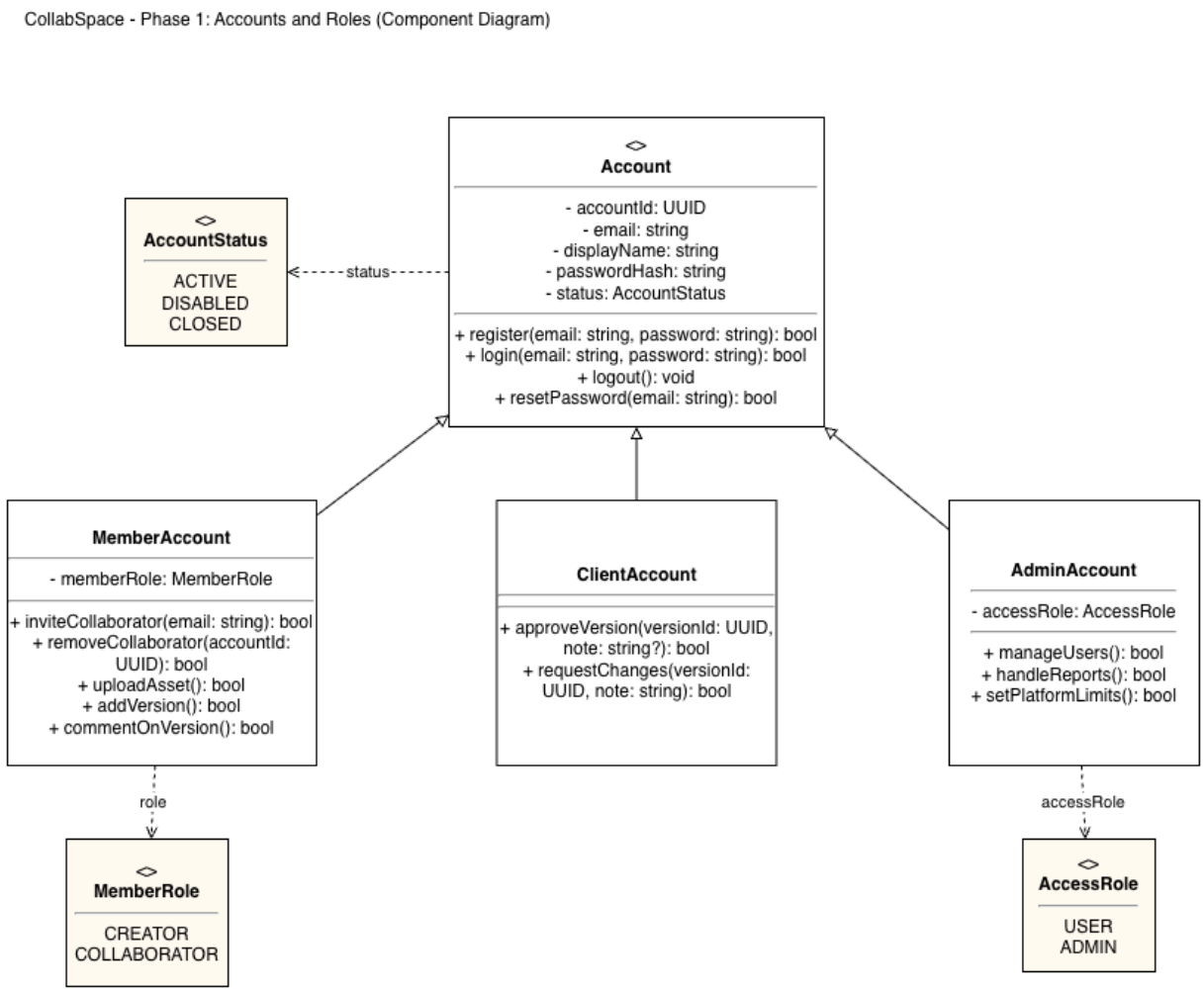


**Diagram 2: Assets + Versions**

This component diagram defines how Assets and AssetVersions are structured in CollabSpace, including core attributes, key operations, and the VersionStatus enumeration used to track review state.

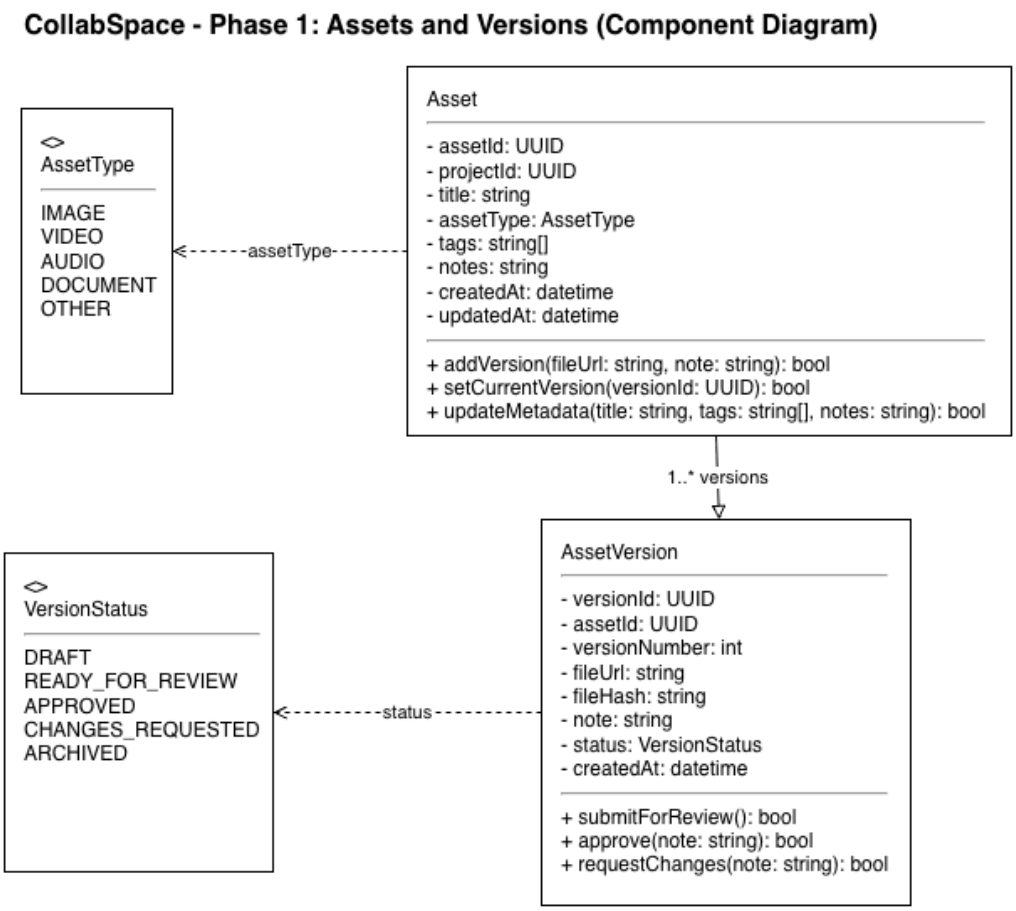


**Diagram 3: PresentationPage + ControlledAccess**

This component diagram models PresentationPage and its Controlled Access mechanism using an AccessPolicy class and AccessMode enumeration to represent public vs restricted viewing.

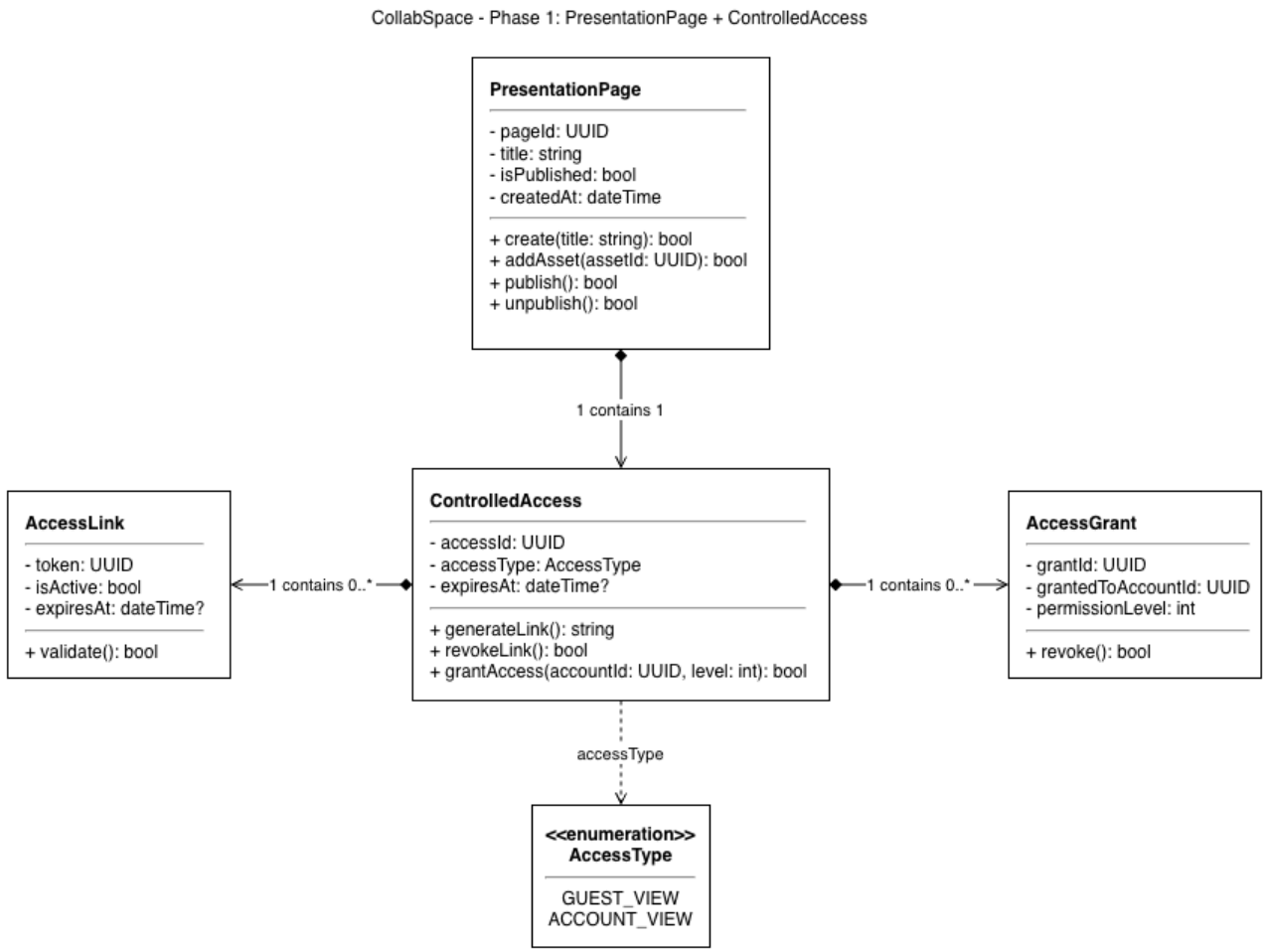


**Diagram 4: Associations**

This association diagram shows the direction of references between major CollabSpace classes, clarifying which objects depend on or point to others.

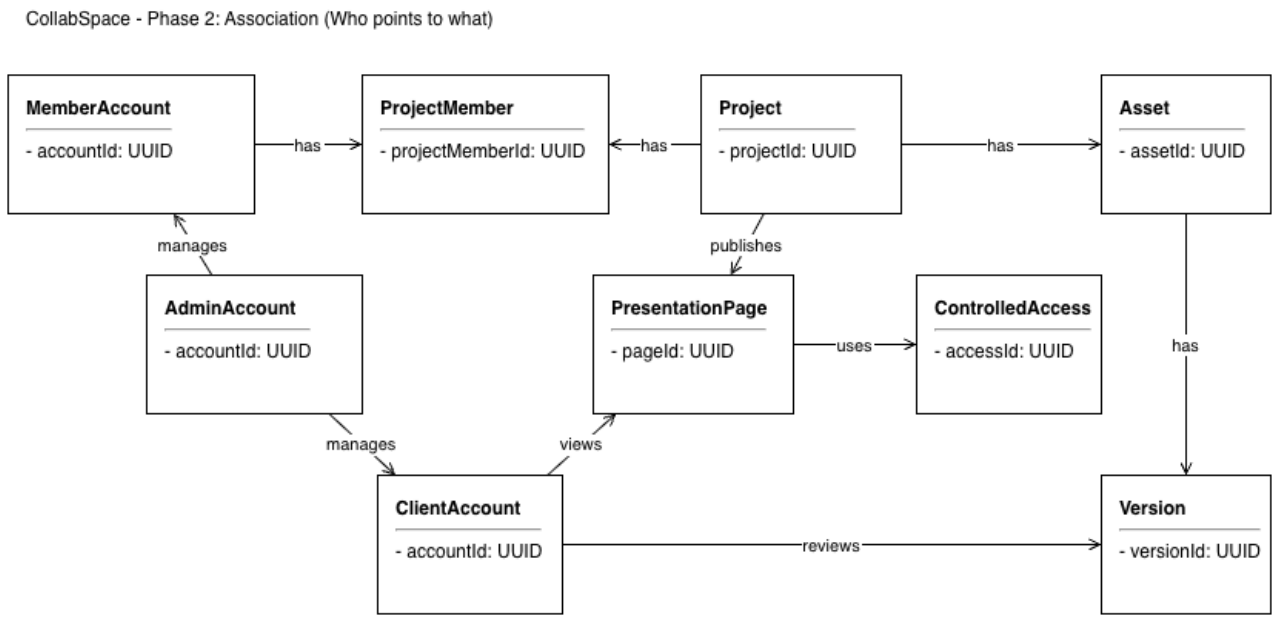
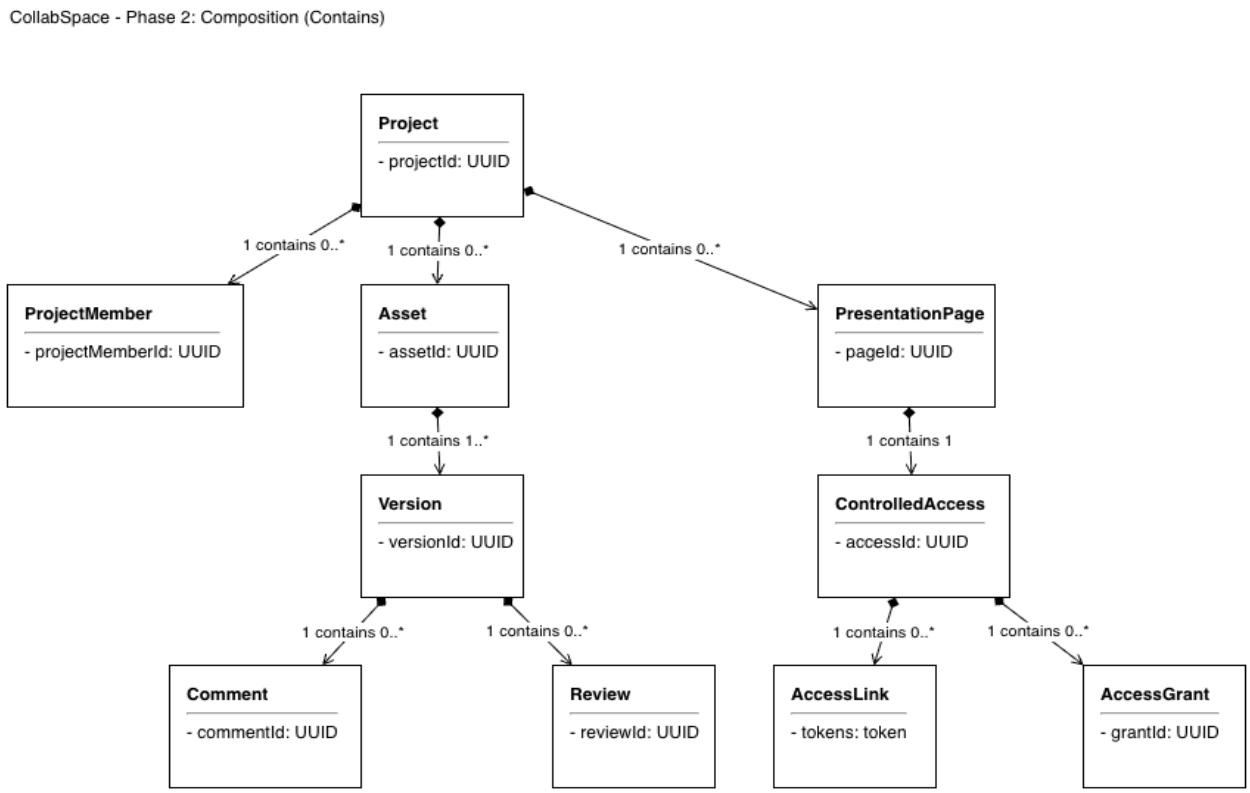


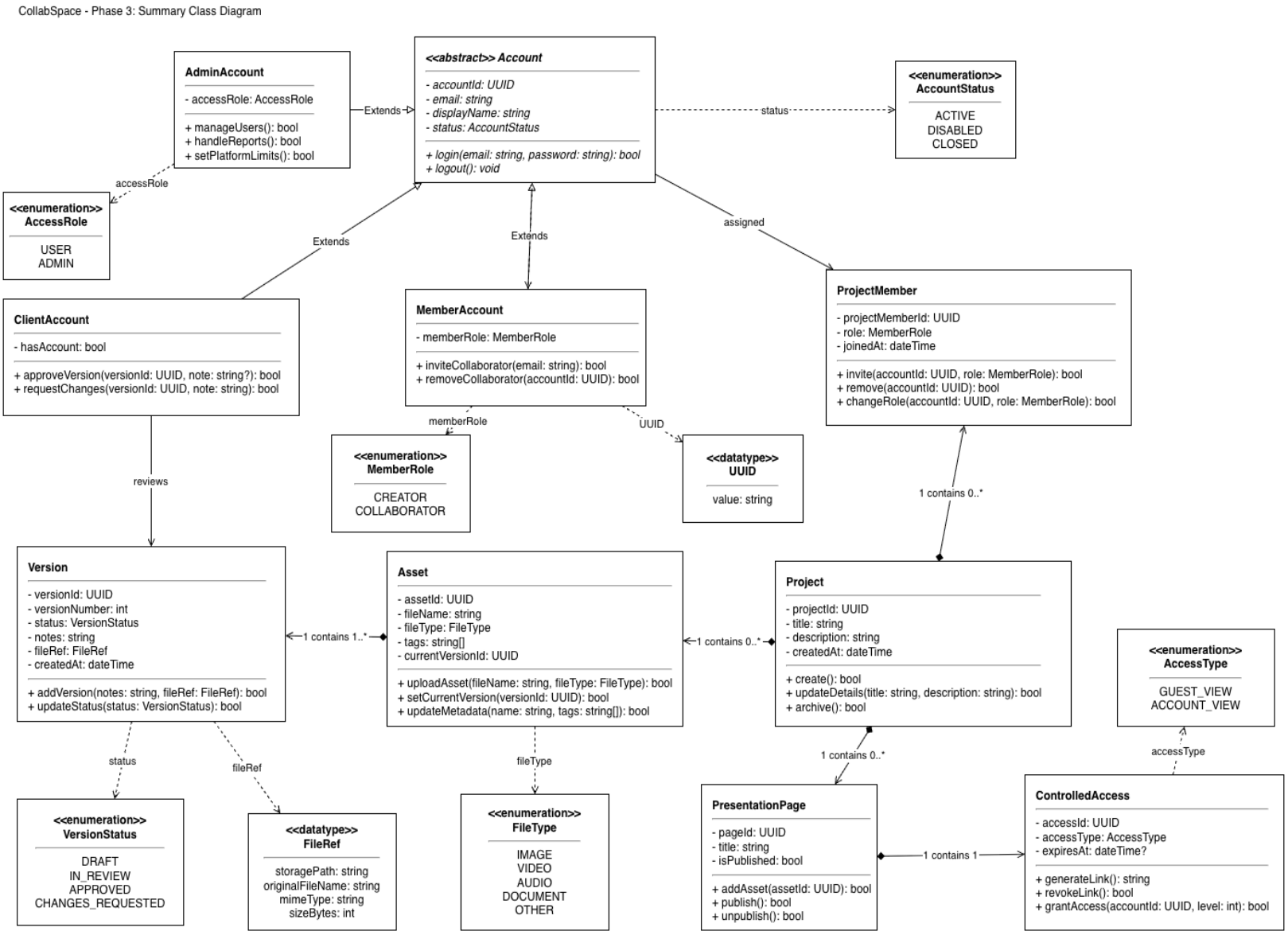
**Diagram 5: Composition**

This composition diagram shows which classes are owned/contained by others (whole–part), such as Project containing Assets and Assets containing Versions.



## Diagram 6: Summary Class Diagram

This summary class diagram combines the core CollabSpace components into one unified view, including key attributes/operations, inheritance (Account types), and major associations/compositions across the system.



# 4.0 System Sequence Diagram and Activity Diagram

## 4.1 Sequence Diagrams

### 4.1.1 System Sequence Diagram 1: Add Version (<> Upload Asset)

**Actor:** MemberAccount (Creator/Collaborator)

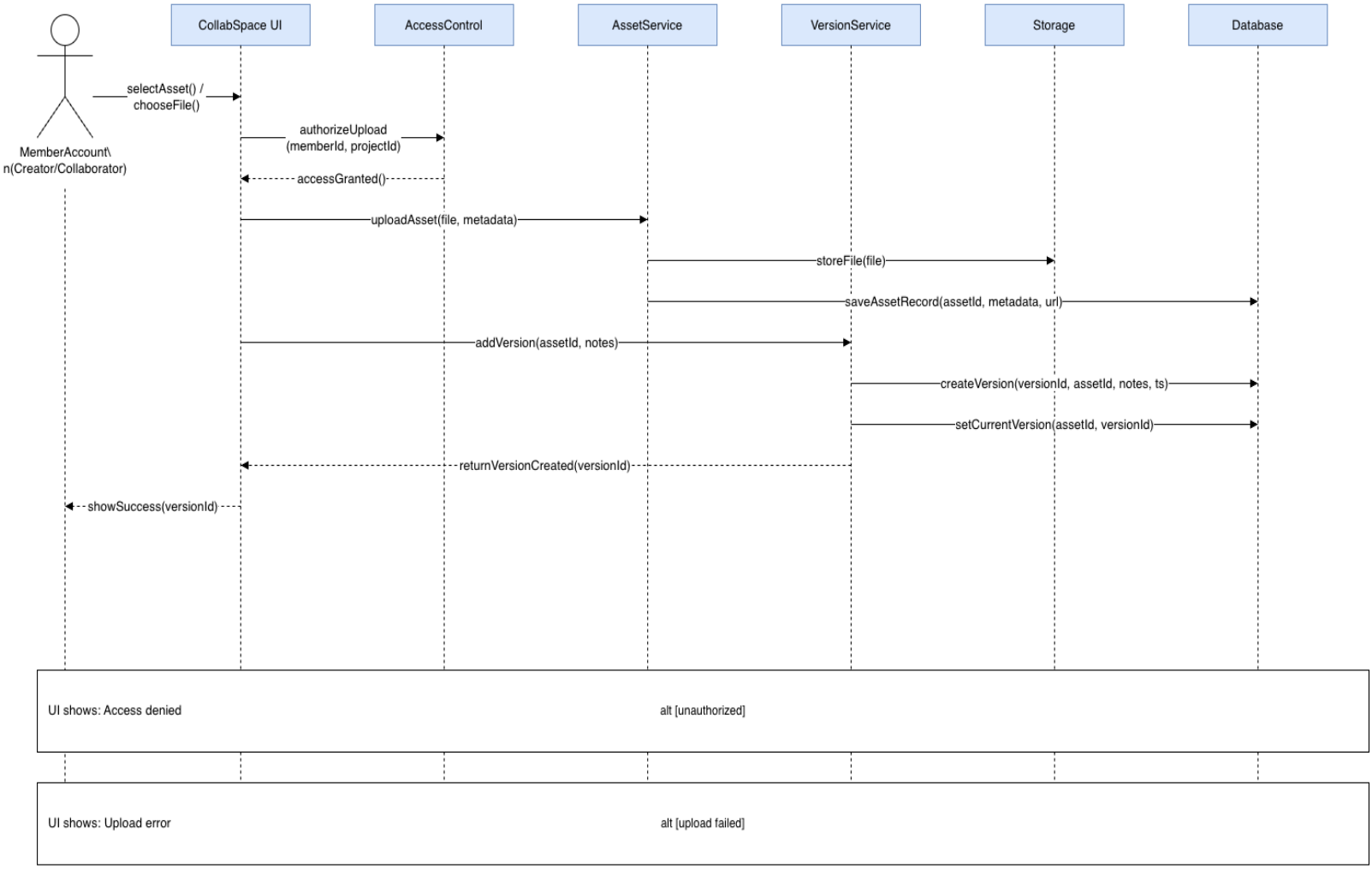
**Objects:** CollabSpace UI, AccessControl, AssetService, VersionService, Storage, Database

1. MemberAccount selects an asset / chooses a file in the CollabSpace UI.
2. CollabSpace UI requests upload authorization from AccessControl using memberId and projectId.
3. If access is granted, CollabSpace UI sends the file and metadata to AssetService to upload the asset.
4. AssetService stores the file in Storage.
5. AssetService saves the asset record (assetId, metadata, storage URL) in the Database.
6. CollabSpace UI requests VersionService to add a new version using the assetId and notes.
7. VersionService creates a new version record (versionId, assetId, notes, timestamp) in the Database.
8. VersionService sets the newly created version as the current version in the Database.
9. VersionService returns the created versionId to the CollabSpace UI.
10. CollabSpace UI displays a success confirmation to MemberAccount.

**Alternatives:**

- If unauthorized, then the UI will show “Access denied.”
- If upload fails, then the UI shows “Upload error.”

Sequence Diagram 1: Add Version



### 4.1.2 System Sequence Diagram 2: Approve Version

**Actor:** Client (GuestViewer/ClientAccount)

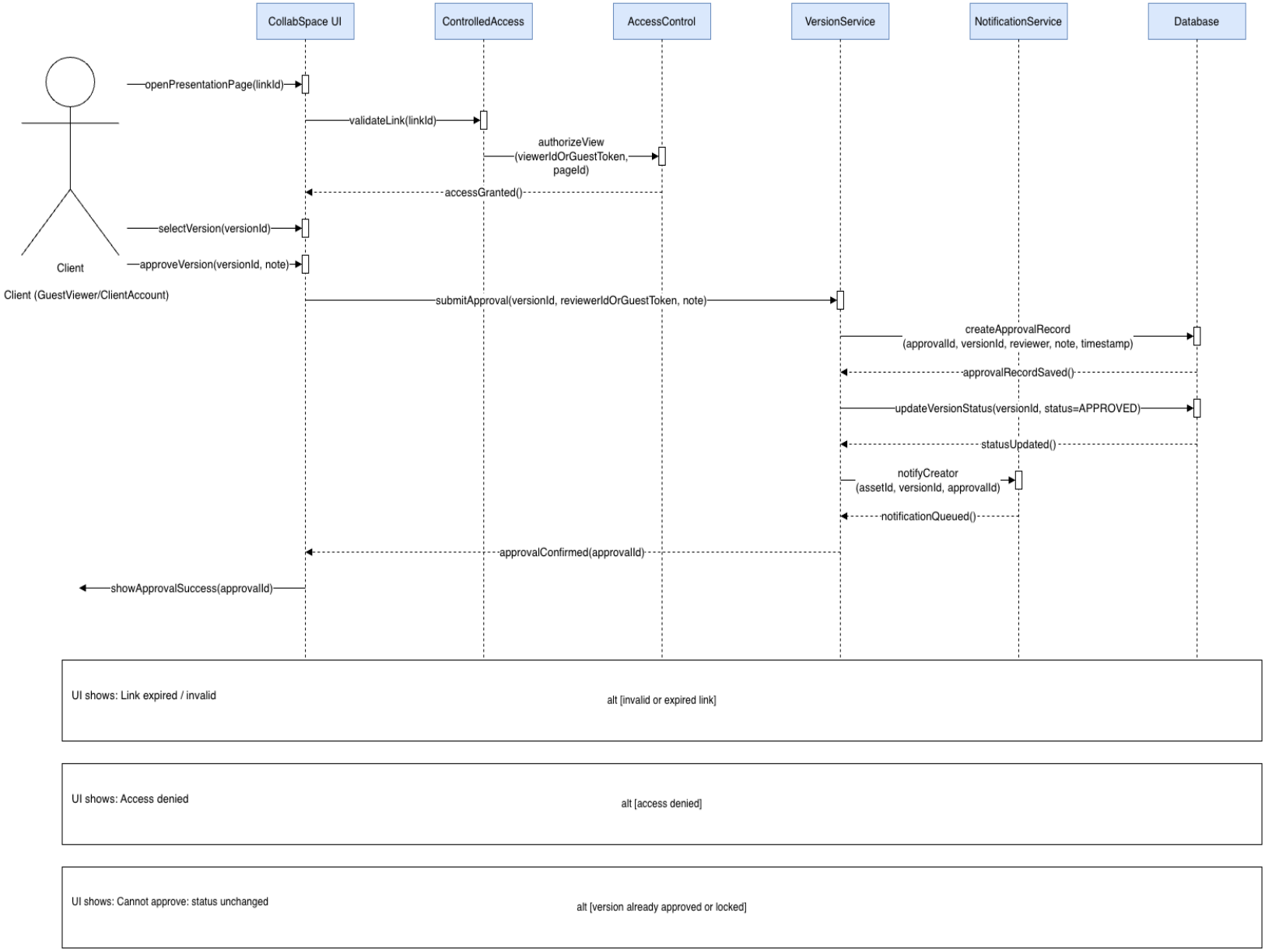
**Objects:** CollabSpace UI, ControlledAccess, AccessControl, VersionService, NotificationService, Database

1. Client opens the presentation page using linkId in the CollabSpace UI.
2. CollabSpace UI asks ControlledAccess to validate the link.
3. ControlledAccess requests AccessControl to authorize the viewer (guest token or account) for the page.
4. If access is granted, AccessControl returns accessGranted() to the CollabSpace UI.
5. Client selects a version to review.
6. Client submits an approval with a note.
7. CollabSpace UI sends the approval request to VersionService.
8. VersionService creates an approval record in the Database.
9. VersionService updates the version status to APPROVED in the Database.
10. VersionService requests NotificationService to notify the Creator.
11. NotificationService returns confirmation that the notification is queued.
12. VersionService returns approval confirmation (approvalId) to the CollabSpace UI.
13. CollabSpace UI shows approval success to the Client.

**Alternatives:**

- If link is invalid or expired, then the UI will show “Link expired / invalid.”
- If access is denied, then the UI will show “Access denied.”
- If version is already approved or locked, then the UI will show “Cannot approve: status unchanged.”

**Sequence Diagram 2: Approve Version**



## 4.2 Activity Diagrams

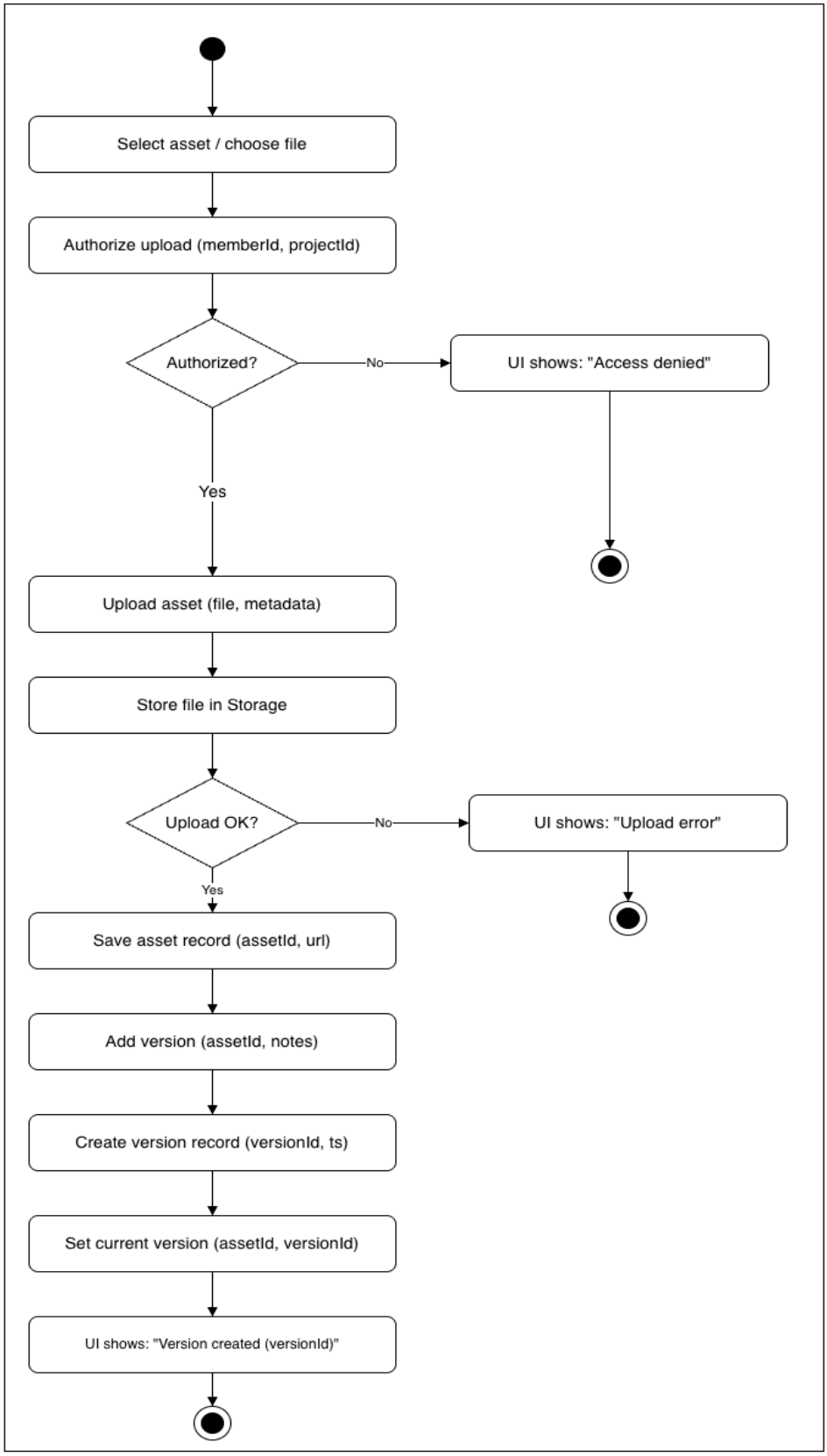
### 4.2.1 Activity Diagram 1: Add Version (includes Upload Asset)

**Initial State:** MemberAccount starts adding a version by selecting a file.

**Final State:** Version is created successfully OR the process ends due to access denial or upload failure.

1. MemberAccount selects an asset / chooses a file.
2. System authorizes the upload request.
3. If not authorized, UI shows “Access denied” and the process ends.
4. If authorized, the system uploads the asset and stores it.
5. If upload fails, UI shows “Upload error” and the process ends.
6. If upload succeeds, the system saves the asset record.
7. The system adds a version and creates a version record.
8. The system sets the new version as the current version.
9. UI shows “Version created” and the process ends.

**Activity Diagram: Add Version (includes Upload Asset)**



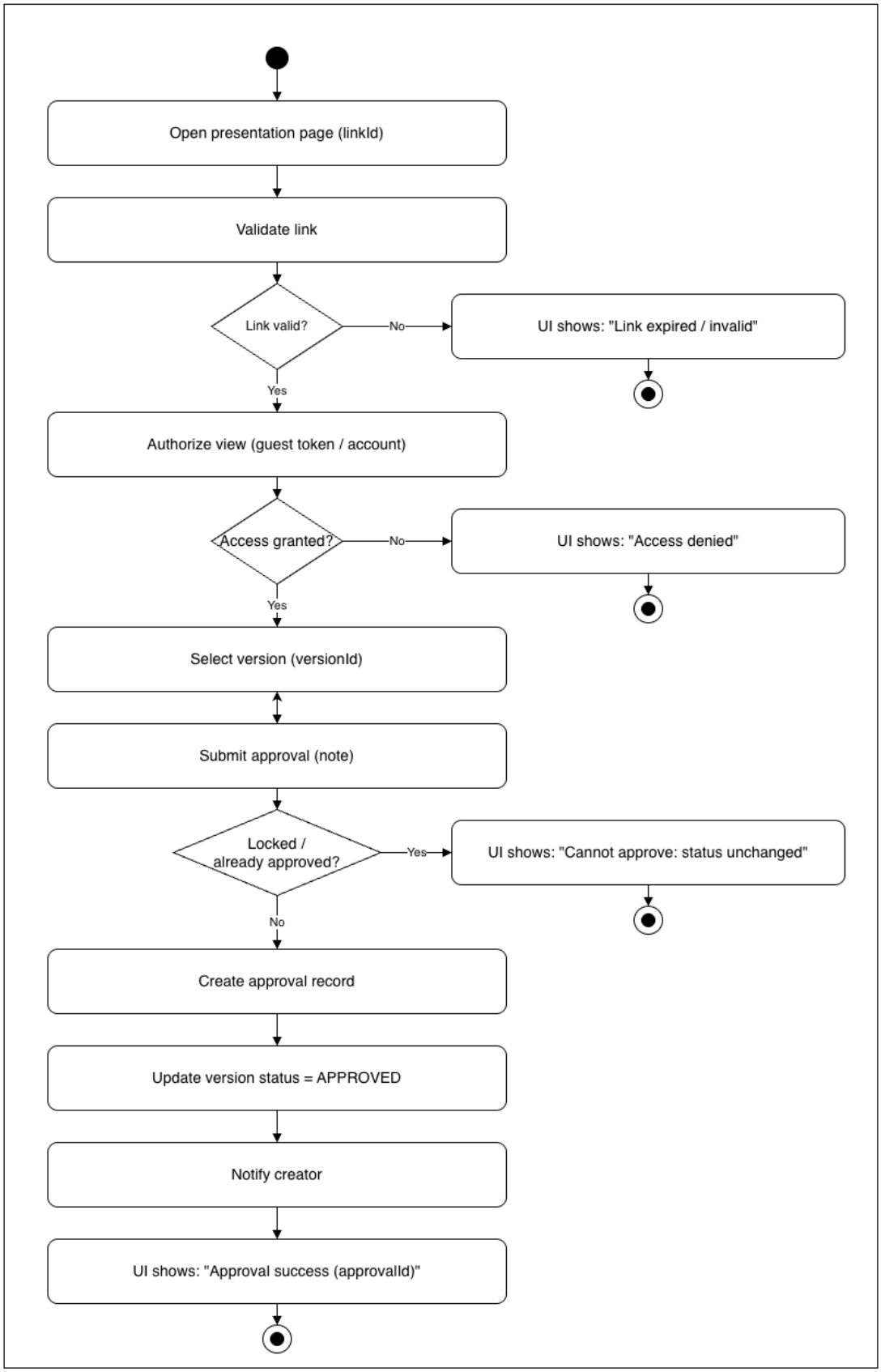
### 4.2.2 Activity Diagram 2: Approve Version

**Initial State:** Client opens the presentation page from a link.

**Final State:** Approval is submitted successfully OR the process ends due to invalid link, access denial, or locked status.

1. Client opens the presentation page (linkId).
2. System validates the link.
3. If link is invalid, UI shows “Link expired / invalid” and the process ends.
4. If valid, system authorizes viewing (guest token / account).
5. If access is denied, UI shows “Access denied” and the process ends.
6. If access is granted, client selects a version.
7. Client submits an approval note.
8. System checks whether the version is locked or already approved.
9. If locked/already approved, UI shows “Cannot approve: status unchanged” and the process ends.
10. If allowed, system creates an approval record.
11. System updates the version status to APPROVED.
12. System notifies the creator.
13. UI shows “Approval success” and the process ends.

**Activity Diagram: Approve Version**



## 5.0 Traceability Matrix

*(*for User Interface Specification, See the additional document within project)*

### System Requirements

| No. | PW (1–5) | Description |
|---|---|---|
| REQ-1 | 5 | Users must be able to create an account and sign in to access their workspace. |
| REQ-2 | 5 | The system must support creating projects to organize creative work. |
| REQ-3 | 5 | Creators must be able to invite/remove collaborators on a project. |
| REQ-4 | 5 | Members must be able to upload assets with basic metadata (title/tags/notes). |
| REQ-5 | 5 | Members must be able to add new versions of an asset and set a current version. |
| REQ-6 | 4 | Members must be able to comment on a version and update its review status. |
| REQ-7 | 4 | Members must be able to search and filter assets within a project. |
| REQ-8 | 5 | Creators must be able to publish a presentation page for selected assets/versions. |
| REQ-9 | 5 | The system must enforce controlled access to presentations (link validation + permissions). |
| REQ-10 | 5 | Clients must be able to approve a version or request changes with optional notes. |
| REQ-11 | 3 | Admins must be able to manage users, reports, and platform limits. |
| REQ-12 | 4 | The system must store project data reliably and support retrieval for viewing/search. |

### Use Cases

| No. | Description |
|---|---|
| UC1 | Create Project |

| No. | Description |
|---|---|
| UC2 | Upload Asset |
| UC3 | Add Version (<> Upload Asset) |
| UC4 | Create Presentation Page |
| UC5 | Approve Version |
| UC6 | Create Account |
| UC7 | Login/Logout |
| UC8 | Manage Collaborators |
| UC9 | Set Current Version |
| UC10 | Update Status |
| UC11 | Comment on Version |
| UC12 | Search/Filter Assets |
| UC13 | Open Presentation Page |
| UC14 | Use Controlled Access |
| UC15 | Request Changes |
| UC16 | Manage Users (Admin) |
| UC17 | Handle Reports (Admin) |
| UC18 | Manage Platform Limits (Admin) |
| UC19 | Store Project Data (System) |
| UC20 | Enforce Access (System) |
| UC21 | Maintain Current Version (System) |
| UC22 | Search/Retrieval (System) |

## Traceability Matrix

| Req't | PW | UC1 | UC2 | UC3 | UC4 | UC5 | UC6 | UC7 | UC8 | UC9 | UC10 | IC11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| REQ1 | 5 | | | | | X | X | | | | | |
| REQ2 | 5 | X | | | | | | | | | | |
| REQ3 | 5 | | X | | | | | | | | | |
| REQ4 | 5 | | | | | | | X | | | | |
| REQ5 | 5 | | | X | | | | | X | | | |
| REQ6 | 4 | | | | | | | | | X | X | |
| REQ7 | 4 | | | | | | | | | | | X |
| REQ8 | 5 | | | | X | | | | | | | |
| REQ9 | 5 | | | | | | | | | | | |
| REQ10 | 5 | | | | | | | | | | | |
| REQ11 | 3 | | | | | | | | | | | |
| REQ12 | 4 | | | | | | | | | | | |
| **MAX PW** | | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 4 | 4 | 4 |
| **TOTAL PW** | | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 4 | 4 | 4 |

| Req't | PW | UC12 | UC13 | UC14 | UC15 | UC16 | UC17 | UC18 | UC19 | UC20 | UC21 | IC22 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| REQ1 | 5 | | | | | | | | | | | |
| REQ2 | 5 | | | | | | | | | | | |
| REQ3 | 5 | | | | | | | | | | | |
| REQ4 | 5 | | | | | | | | | | | |
| REQ5 | 5 | | | | | | | | | | X | |
| REQ6 | 4 | | | | | | | | | | | |
| REQ7 | 4 | | | | | | | | | | | X |
| REQ8 | 5 | | | | | | | | | | | |
| REQ9 | 5 | X | X | | | | | | | X | | |
| REQ10 | 5 | | | X | X | | | | | | | |
| REQ11 | 3 | | | | | X | X | X | | | | |
| REQ12 | 4 | X | | | | | | | X | | | X |
| **MAX PW** | | 5 | 5 | 5 | 5 | 3 | 3 | 3 | 4 | 5 | 5 | 4 |
| **TOTAL PW** | | 9 | 5 | 5 | 5 | 3 | 3 | 3 | 4 | 5 | 5 | 8 |